


```
from numpy import fill_diagonal
# plot average monthly temperate derived from from dally medians for each location
fig = go.Figure()
for location in df2_pivot.index:
    fig.add_trace(go.Scatter(x=df2_pivot.loc[location, :].index,
                             y=df2_pivot.loc[location, :].values,
                             mode='lines', name=location,
                             opacity=0.8, linewidth=1)
    ))

# Change chart background color
fig.update_layout(dict(plot_bgcolor = 'white'), showlegend=True)

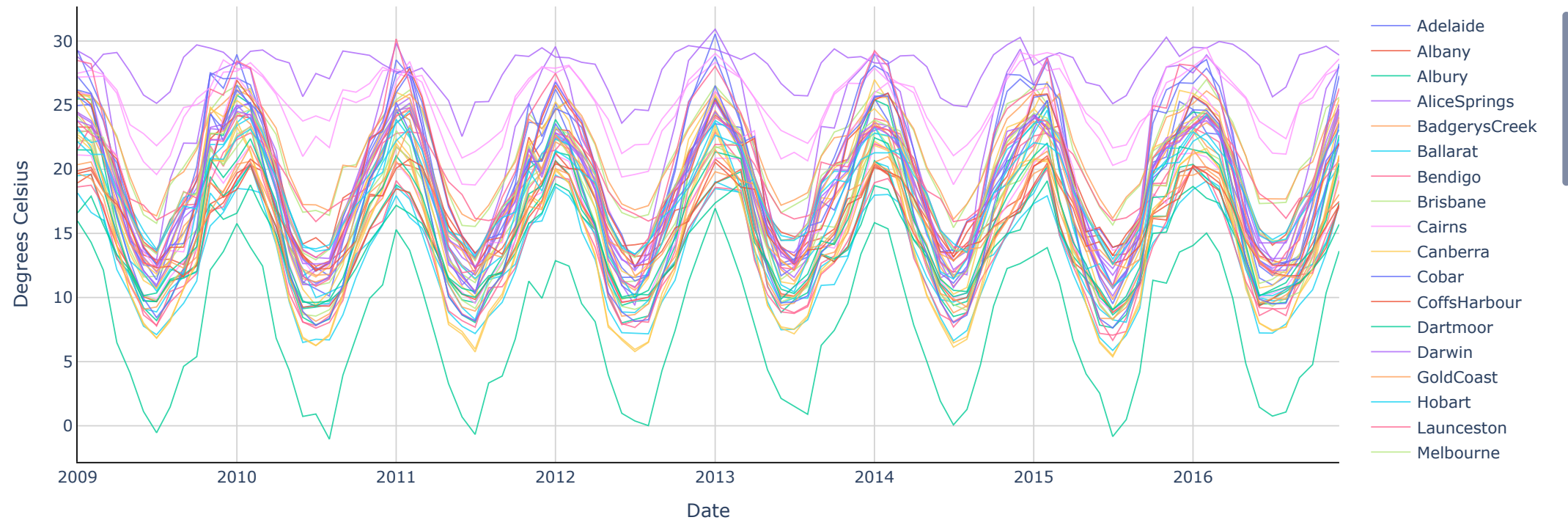
# Update axes lines
fig.update_xaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Date'
                 )

fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Degrees Celsius'
                 )

# Set figure title
fig.update_layout(title=dict(text="Average Monthly Temperatures", font=dict(color='black'))))

fig.show()
```

Average Monthly Temperatures



```
import numpy as np
```

```
def shaping(datain, timestep):
    # Convert input dataframe to array and flatten
    arr = datain.to_numpy().flatten()

    X_list, y_list = [], []
    for i in range(len(datain.columns) - 2 * timestep + 1):
        X_list.append(arr[i:i+timestep])
        y_list.append(arr[i+timestep:i+2*timestep])

    # Reshape input and target arrays
    X_out = np.array(X_list).reshape(-1, timestep, 1)
    y_out = np.array(y_list).reshape(-1, timestep, 1)
    return X_out, y_out
```

```
#### step 1 - specify parameters ####
timestep=18
location='Canberra'
```

```
##### Step 2 - prepare data
# split data into train & test dataframes
df_train = df2_pivot.iloc[:, :-2*timestep].copy()
df_test = df2_pivot.iloc[:, -2*timestep:].copy()
```

```
# Select one location
dfloc_train = df_train[df_train.index==location].copy()
dfloc_test = df_test[df_test.index==location].copy()
```

```
# Use previously defined shaping function to reshape the data for LSTM
X_train, y_train = shaping(datain=dfloc_train, timestep=timestep)
X_test, y_test = shaping(datain=dfloc_test, timestep=timestep)
```

```
from tensorflow import keras # for building Neural Networks
from keras.models import Sequential # for creating a linear stack of layers for our Neural Network
from keras import Input # for instantiating a keras tensor
from keras.layers import Bidirectional, GRU, RepeatVector, Dense, TimeDistributed # for creating layers inside the Neural Network
```

```
##### Step 3 - Specify the structure of a Neural Network
model = Sequential(name="GRU-Model")
# Input Layer - need to specify the shape of inputs
model.add(Input(shape=(X_train.shape[1],X_train.shape[2]), name="Input-Layer"))
# Encoder Layer
model.add(Bidirectional(GRU(units=32, activation='tanh', recurrent_activation='sigmoid', stateful=False), name="Hidden-GRU-Encoder-Layer"))
# Repeat Vector
model.add(RepeatVector(y_train.shape[1], name="Repeat-Vector-Layer"))
# Decoder Layer
model.add(Bidirectional(GRU(units=32, activation='tanh', recurrent_activation='sigmoid', stateful=False, return_sequences=True), name="Hidden-GRU-Decoder-Layer"))
# Output Layer, Linear(x) = x
model.add(TimeDistributed(Dense(units=1, activation='linear'), name="Output-Layer"))
```

```
##### Step 4 - Compile the model
model.compile(
    optimizer='adam', loss='mean_squared_error',
    metrics=['mean_squared_error', 'mean_absolute_error']
)
```

```
history = model.fit(X_train, y_train,
                    batch_size=1, epochs=150,
                    validation_split=0.2, shuffle=True, verbose=0)
```

```
##### Step 5 - Use model to make predictions
# Predict results on test data
pred_test = model.predict(X_test)
```

1/1 ————— 1s 1s/step

```
##### Step 7 - Print Performance Summary
print("")
print('..... Model Summary .....')
model.summary() # print model summary
print("")
print('..... Weights and Biases .....')
print("Too many parameters to print but you can use the code provided if needed")
print("")
for layer in model.layers:
    print(layer.name)
    for item in layer.get_weights():
        print(" ", item)
print("")
```

[Show hidden output](#)

```
# Print the last value in the evaluation metrics contained within history file
print('..... Evaluation on Training Data .....')
for item in history.history:
    print("Final", item, ":", history.history[item][-1])
print("")
```

```
..... Evaluation on Training Data .....
Final loss : 0.992263674736823
Final mean_absolute_error : 0.7795861330813249
Final mean_squared_error : 0.992263674736823
Final val_loss : 3.344552993774414
Final val_mean_absolute_error : 1.5598778388028142
Final val_mean_squared_error : 3.344552993774414
```

```
# Evaluate the model on the test data using "evaluate"
print('..... Evaluation on Test Data .....')
results = model.evaluate(X_test, y_test)
print("")
```

1/1 ————— 1s 1s/step - loss: 3.7289 - mean_absolute_error: 1.6688 - mean_squared_error: 3.7289

```
# Plot average monthly temperatures (actual and predicted) for test (out of time) data
fig = go.Figure()

# Trace for actual temperatures
fig.add_trace(go.Scatter(x=np.array(dfloc_test.columns),
                         y=np.array(dfloc_test.values).flatten(),
                         mode='lines',
                         name='Average Monthly Temperatures - Actual (Test)',
                         opacity=0.8,
                         linewidth=1)
))

# Trace for predicted temperatures
fig.add_trace(go.Scatter(x=np.array(dfloc_test.columns[-timestep:]),
                         y=pred_test.flatten(),
                         mode='lines',
                         name='Average Monthly Temperatures - Predicted (Test)',
                         opacity=0.8,
                         linewidth=1)
))

# Change chart background color
fig.update_layout(dict(plot_bgcolor = 'white'))

# Update axes lines
fig.update_xaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Year-Month'
                 )

fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Degrees Celsius'
                 )

# Set figure title
fig.update_layout(title=dict(text="Average Monthly Temperatures", font=dict(color='black')),
                  legend=dict(orientation="h", yanchor="bottom", y1=0.8, xanchor="right", x=1)
                  )

fig.show()
```

Average Monthly Temperatures

